МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Отчет по лабораторной работе №4

по курсу «Операционные системы»

Тема: «Межпроцессное взаимодействие через отображаемую память»

Вариант 18

Выполнил(а): А. Е. Пронина
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Цель работы

Изучить работу с файловыми объектами и организацию обмена данными между процессами с помощью отображаемой памяти (file mapping). Реализовать передачу строк через `mmap()`, синхронизировать доступ POSIX-семафорами и обеспечить освобождение ресурсов при штатном завершении.

Постановка задачи

Для варианта 18 требуется реализовать родительский процесс и два дочерних процесса. Родитель считывает имена двух выходных файлов и строки стандартного ввода. Нечётные строки данных направляются первому ребёнку, чётные — второму. Пустая строка также имеет номер и участвует в чередовании.

Каждый дочерний процесс удаляет из полученных строк английские гласные и записывает результат в свой файл. В исследуемом коде удаляются символы `a, e, i, o, u` и их прописные варианты. Символ `y` и русские гласные данным фильтром не удаляются.

Для ЛР № 4 обмен организуется через две области общей памяти. Дочерние процессы создаются с помощью `fork()`, после чего выполняется `execvp()` программы `child`. После EOF родитель передаёт признак завершения, ожидает детей и освобождает объекты общей памяти.

Архитектура решения

Программа состоит из двух исполняемых файлов: `parent` и `child`. Каждый ребёнок обслуживает собственный канал с единственным буфером.

Стандартный ввод $\longrightarrow$ `parent`
нечётные строки $\longrightarrow$ память 1 $\longrightarrow$ `child 1` $\longrightarrow$ файл 1
чётные строки $\longrightarrow$ память 2 $\longrightarrow$ `child 2` $\longrightarrow$ файл 2
В каждой области: данные, флаги и два POSIX-семафора

Служебные сообщения дочерних процессов выводятся в `stderr`. Результат фильтрации направляется в `stdout`, который ребёнок перенаправляет в выходной файл через `open()` и `dup2()`.

Общая память и протокол обмена

Структура данных

В `shared.h` объявлена следующая структура:

```
1 #define BUFFER_SIZE 4096
2
3 typedef struct {
```

```
4       sem_t can_write;
5       sem_t can_read;
6       size_t length;
7       int end_of_line;
8       int finished;
9       char text[BUFFER_SIZE];
10  } SharedData;
```

`length` задаёт число передаваемых байтов; `end_of_line` отмечает последнюю часть строки; `finished` сообщает об окончании обмена. Буфер не обязан содержать завершающий нулевой байт: фильтр и `fwrite()` используют явную длину.

Объекты именуются по шаблону `/lab4_<PID>_1` и `/lab4_<PID>_2`. В трассировке размер каждого отображения равен 4176 байтам: это размер всей структуры в данной сборке, а не только массива `text`. Размер полезной порции равен 4096 байтам.

Синхронизация

Оба семафора размещаются непосредственно в общей памяти. Параметр `pshared=1` при `sem_init()` разрешает межпроцессное использование. Начальные значения: `can_write=1`, `can_read=0`.

1. Родитель ожидает `can_write`, записывает данные, длину и флаги, затем выполняет `sem_post(can_read)`.
2. Ребёнок ожидает `can_read`, обрабатывает порцию и записывает результат. После этого он выполняет `sem_post(can_write)`.
3. Следующая порция перезаписывает буфер только после разрешения ребёнка. Между частями одной строки перевод строки не добавляется.

При EOF отправляется сообщение с `finished=1`. Получив его, ребёнок освобождает семафор записи и завершает цикл.

Используемые механизмы ОС

Отображение памяти

`shm_open()` создаёт или открывает именованный объект POSIX shared memory. `ftruncate()` задаёт его размер, а `mmap()` отображает объект в виртуальное адресное пространство процесса. После успешного отображения файловый дескриптор закрывается; само отображение продолжает существовать.

Процессы имеют отдельные виртуальные адресные пространства. Поэтому один объект может находиться по разным виртуальным адресам у родителя и ребёнка. Флаг `MAP_SHARED` обеспечивает совместный доступ к изменениям объекта. При `MAP_PRIVATE` изменения через отображение не использовались бы как общий канал записи между процессами.

Одного MAP_SHARED недостаточно для согласования доступа: семафоры задают порядок заполнения и чтения буфера. Они предотвращают чтение частично подготовленного сообщения и перезапись ещё не обработанных данных.

Распределение и обработка строк

В parent.c строка читается функцией getline(), завершающий \n исключается из полезных данных. Выбор ребёнка выполняется по остатку от деления счётчика line_number на 2. Счётчик начинается с нуля, поэтому первая строка направляется ребёнку 1.

send_line() разбивает строку на части до 4096 байт. Цикл do ... while отправляет сообщение даже для пустой строки: длина равна нулю, а end_of_line=1. Ребёнок в этом случае записывает один перевод строки.

remove_vowels() проходит по данным один раз и уплотняет сохраняемые символы в том же буфере. Для строки длины n обработка занимает $O(n)$ времени и $O(1)$ дополнительной памяти фильтра. Память родителя для getline() зависит от длины строки.

Ключевые фрагменты исходного кода

Ниже приведены ключевые фрагменты программы. Обозначение /* ... */ показывает сокращённые участки обработки ошибок.

Создание области общей памяти

Основные операции функции create_shared() из shared.c:

```
int fd = shm_open(name, O_CREAT | O_EXCL | O_RDWR, 0600);
/* ... */
if (ftruncate(fd, sizeof(SharedData)) == -1) {
    perror("ftruncate");
    cleanup_shared(name, fd, MAP_FAILED, 0, 0);
    return -1;
}
void* ptr = mmap(NULL, sizeof(SharedData),
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
/* ... */
SharedData* shared = (SharedData*)ptr;
if (sem_init(&shared->can_write, 1, 1) == -1) {
    /* ... */
}
if (sem_init(&shared->can_read, 1, 0) == -1) {
    /* ... */
}
```

O_EXCL предотвращает открытие уже существующего объекта под видом нового. Функция cleanup_shared() освобождает успешно созданные части ресурса при ошибке и удаляет имя объекта.

Публикация сообщения

В `send_message()` родитель ожидает `can_write` через `sem_timedwait()` с очередным сроком ожидания через одну секунду. После `EINTR` или `ETIMEDOUT` выполняется `waitpid(..., WNOHANG)`: обнаруженное завершение ребёнка прерывает передачу. После успешного ожидания выполняется:

```
if (length > 0) memcpy(child->data->text, text, length);
child->data->length = length;
child->data->end_of_line = end_of_line;
child->data->finished = finished;
if (sem_post(&child->data->can_read) == -1) {
    perror("sem_post");
    return -1;
}
```

Передача строки частями

Функция `send_line()` из `parent.c` приведена полностью:

```
static int send_line(ChildInfo* child,
                     const char* line, size_t length) {
    size_t offset = 0;
    do {
        size_t part = length - offset;
        if (part > BUFFER_SIZE) part = BUFFER_SIZE;
        int last = (offset + part == length);
        if (send_message(child, line + offset,
                         part, last, 0) == -1)
            return -1;
        offset += part;
    } while (offset < length);
    return 0;
}
```

Флаг `last` обеспечивает один перевод строки после всех её частей. Разбиение производится по байтам; английские гласные в фильтре представлены однобайтовыми ASCII-символами.

Приём и обработка в дочернем процессе

Ниже показаны основные действия цикла из `child.c`. Проверки ошибок вывода сокращены.

```
if (wait_for_data(&data->can_read) == -1) {
    result = 1;
    break;
}
if (data->finished) {
    if (sem_post(&data->can_write) == -1) {
        perror("sem_post");
        result = 1;
    }
    break;
```

```
}
/* ... */
size_t length = remove_vowels(data->text, data->length);
if (length > 0) {
    if (fwrite(data->text, 1, length, stdout) != length) {
        /* ... */
    }
}
if (data->end_of_line) {
    if (fputc('\n', stdout) == EOF) { /* ... */ }
}
if (sem_post(&data->can_write) == -1) { /* ... */ }
```

Фильтрация и освобождение ресурсов

Функция из `filter.c` выполняет фильтрацию без выделения второго буфера:

```
size_t remove_vowels(char* text, size_t length) {
    size_t j = 0;
    for (size_t i = 0; i < length; i++) {
        if (!is_vowel(text[i])) text[j++] = text[i];
    }
    return j;
}
```

`is_vowel()` сравнивает символ с `a,e,i,o,u,A,E,I,O,U`. Цифры, знаки пунктуации и согласные сохраняются в прежнем порядке.

Перед очисткой родитель ожидает завершения каждого ещё не собранного ребёнка через `waitpid()`; ожидание повторяется при `EINTR`. Проверяются `WIFEXITED(status)` и `WEXITSTATUS(status)`. Затем вызывается `destroy_shared()` для каждой созданной области.

В `destroy_shared()` последовательно выполняются следующие вызовы (между ними в исходнике находятся проверки результата):

```
sem_destroy(&data->can_write)
sem_destroy(&data->can_read)
munmap(data, sizeof(SharedData))
shm_unlink(name)
```

Ребёнок снимает собственное отображение через `close_shared()`, где вызывается `munmap()`. `munmap()` освобождает отображение процесса, а `shm_unlink()` удаляет имя объекта. Это разные операции.

Сборка и запуск

Проект использует C11 и CMake. `parent` собирается из `parent.c` и `shared.c`; `child` — из `child.c`, `filter.c` и `shared.c`. Обе цели связаны с `Threads::Threads`; для GNU/Clang включены `-Wall -Wextra -Wpedantic`.

```
cmake -S . -B build
cmake --build build
```

```
cd build
./parent
```

Запуск выполняется из `build`, поскольку путь `./child` в `execvp()` задан относительно текущего каталога.

Результаты функционального запуска

При интерактивном запуске родитель имел PID 52126, дети — PID 52159 и 52160. Были указаны файлы `out1` и `out2`. Оба ребёнка вывели сообщения `started` и `finished`.

Результаты обработки входных строк приведены в таблице. Обозначение ∅ означает пустую строку.

№	Файл	Входная строка	Результат
1	`out1`	`aaaaaaa`	∅
2	`out2`	∅	∅
3	`out1`	∅	∅
4	`out2`	`akdlkasldkalk`	`kdlksldklk`
5	`out1`	`paspofk`	`pspfk`
6	`out2`	`slkfslakjf`	`slkfslkjf`
7	`out1`	Строка со скобками, буквами и цифрами (см. ниже)	См. точный результат ниже
8	`out2`	`gggg`	`gggg`
9	`out1`	`;amibfws34`	`;mbfws34`

Для строки 7 исходное и полученное значения:

```
[]][[weflkcaonijefj98549jos[apvk[pmdbna
[]][[wflkcnjfj98549js[pvk[pmdbn
```

Файл `out1` содержит 49 байт и 5 переводов строки, `out2` — 27 байт и 4 перевода строки. Девять входных строк распределены между процессами в соотношении 5/4.

Запуск показывает обработку пустых строк, удаление строчных английских гласных и сохранение согласных, цифр и специальных символов.

Анализ трассировки strace

Порядок запуска

Для исследования системных вызовов использована команда:

```
printf 'trace1\ntrace2\nhello\nworld\n' | strace -f \
  -e trace=process,memory,openat,close,ftruncate,futex,unlink,unlinkat \
  -o trace.log ./parent
```

В файлах `trace1` и `trace2` получены соответственно `hll\n` (4 байта) и `wrld\n` (5 байт). Гласные из строк `hello` и `world` удалены.

Создание и подключение общей памяти

Ниже приведены системные вызовы, связанные с созданием и подключением общей памяти.

```
54251 openat(AT_FDCWD, "/dev/shm/lab4_54251_1", O_RDWR|O_CREAT|O_EXCL|
   O_NOFOLLOW|O_CLOEXEC, 0600) = 3
54251 ftruncate(3, 4176)                = 0
54251 mmap(NULL, 4176, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0
   x7171c1138000
54251 openat(AT_FDCWD, "/dev/shm/lab4_54251_2", O_RDWR|O_CREAT|O_EXCL|
   O_NOFOLLOW|O_CLOEXEC, 0600) = 3
54251 ftruncate(3, 4176)                = 0
54251 mmap(NULL, 4176, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0
   x7171c1136000
54252 openat(AT_FDCWD, "/dev/shm/lab4_54251_1", O_RDWR|O_NOFOLLOW|O_CLOEXEC
   ) = 3
54252 mmap(NULL, 4176, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0 <unfinished
   ...>
54252 <... mmap resumed>)               = 0x7757db6ae000
54253 openat(AT_FDCWD, "/dev/shm/lab4_54251_2", O_RDWR|O_NOFOLLOW|O_CLOEXEC
   ) = 3
54253 mmap(NULL, 4176, PROT_READ|PROT_WRITE, MAP_SHARED, 3, 0) = 0
   x7f27a4763000
```

Родитель 54251 создал два объекта, а процессы 54252 и 54253 открыли по одному объекту каждый. Для всех четырёх отображений указан `MAP_SHARED`. Разные адреса одного объекта у родителя и ребёнка подтверждают независимость их виртуальных адресных пространств.

Синхронизация и штатная очистка

Выдержка с ожиданием родителя и пробуждением со стороны ребёнка:

```
54251 futex(0x7171c1138000, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 0, {
   tv_sec=1794436131, tv_nsec=979383363}, FUTEX_BITSET_MATCH_ANY <
   unfinished ...>
54252 futex(0x7757db6ae000, FUTEX_WAKE, 1 <unfinished ...>
54251 <... futex resumed>)              = 0
54252 <... futex resumed>)              = 1
```

Адреса соответствуют началу отображений первого объекта, где в `SharedData` расположен `can_write`. С учётом исходного кода эти операции согласуются с ожиданием и освобождением семафора. Не каждый вызов POSIX-семафора требует отдельного системного вызова, поэтому число операций обмена по числу строк `futex` не определяется.

Завершающая часть трассы содержит следующие выбранные строки:

```
54252 munmap(0x7757db6ae000, 4176 <unfinished ...>
54252 <... munmap resumed>)             = 0
54253 munmap(0x7f27a4763000, 4176)      = 0
54252 +++ exited with 0 +++
54253 +++ exited with 0 +++
54251 munmap(0x7171c1138000, 4176)      = 0
54251 unlink("/dev/shm/lab4_54251_1")   = 0
```

```
54251 munmap(0x7171c1136000, 4176)      = 0
54251 unlink("/dev/shm/lab4_54251_2")   = 0
54251 +++ exited with 0 +++
```

Также в трассе присутствуют уведомления `SIGCHLD` для обоих детей со значением `si_status=0`. Дети сняли собственные отображения и завершились с кодом 0. Родитель снял оба отображения, успешно удалил оба имени через `unlink()` и завершился с кодом 0.

При штатном завершении освобождены отображения и удалены имена обоих созданных объектов общей памяти.

Итоги проверки и выводы

Проверяемое свойство	Основание и результат
Два дочерних процесса	В интерактивном запуске указаны PID 52159 и 52160; в трассе видны процессы 54252 и 54253.
Чередование строк и фильтрация	Файлы `out1/out2`: 5 и 4 строки, результаты соответствуют вводу.
Пустые строки и строка из гласных	Пустые результаты сохранены как отдельные строки.
Две области `MAP_SHARED`	Трасса содержит создание и подключение объектов по 4176 байт.
Завершение и очистка в трассируемом запуске	Три кода завершения 0; успешные `munmap()` и два `unlink()`.

В работе реализован обмен строками между родителем и двумя дочерними процессами через именованные объекты общей памяти. Семафоры `can_write` и `can_read` обеспечивают последовательную передачу порций в каждом канале. Разделение признаков конца строки и конца обмена позволяет сохранять пустые строки и передавать строки частями.

Результаты запуска подтверждают фильтрацию строк и работу двух каналов. Трассировка показывает создание общей памяти, совместные отображения, операции ожидания и пробуждения, а также штатное освобождение ресурсов.